Adaptive Candidate-Budget Beam Search for Folded-Subgraph Local Community Detection

Author Name
Department and Institution
City, Country
email@example.com

Abstract - Local community detection seeks the community containing a queried seed node without requiring a global partition of the network. The folded-subgraph (FG) method improves local modularity expansion by grouping tightly connected neighbors before community growth. Its modularity stage, however, commits to a single candidate at each iteration. A locally favorable candidate can therefore prevent a later, more suitable expansion. This paper introduces a validation-tuned adaptive candidate-budget beam-search extension of FG. The extension preserves the folded-subgraph construction, density rule, modularity equations, non-negative expansion constraint, and relaxation stage. It changes only the search policy: several partial expansion paths are retained, while a validated local budget limits the number of expensive folded candidates examined at each state. On the generated LFR benchmark used by this implementation, the method was selected on validation graphs with a beam width of two, maximum depth five, learned path weight 0.10, and candidate budget 64. Across all 3,000 eligible seeds in held-out LFR6-LFR9 graphs, mean F1 increased from 0.621031 for the original implementation to 0.713990 for the adaptive method. Mean per-seed runtime was 180.392 ms for the adaptive method and 1,323.625 ms for the original method in this Java implementation. The improvement was not universal: LFR6 favored the original method, whereas the adaptive method improved LFR7-LFR9. The results identify irreversible greedy expansion as a substantive weakness while avoiding a claim of universal dominance.

Keywords: local community detection, folded subgraph, modularity expansion, beam search, LFR benchmark, graph mining.

Introduction

Community detection is commonly formulated as a global graph-partitioning problem. In many applications, however, the user supplies one node and needs only the local community surrounding that node. Local methods can reduce memory and computation, but they must make decisions with incomplete context and are sensitive to boundary edges.

The folded-subgraph method of Zhang and Bi groups neighboring nodes into dense local structures before modularity expansion [zhang2025fg]. This design is attractive because it combines a local representation with a modularity criterion and a relaxation step for noisy boundaries. The method nevertheless uses a greedy modularity decision: after evaluating the available folded subgraphs, it permanently selects the candidate with the greatest immediate gain. That decision is not necessarily the best decision for the complete expansion path.

This paper studies that failure mode and proposes a search-based extension. The central idea is deliberately narrow. The original graph statistics and folding rules are retained, but the modularity stage is allowed to preserve a small number of alternatives. A learned local compatibility score is used only as a secondary path-ranking term. To control the extra cost, candidates are first ordered by inexpensive local connectivity information and only a validation-tuned prefix is folded exactly.

The contributions are as follows: - We identify irreversible candidate commitment as a distinct algorithmic weakness of the FG modularity stage. - We implement a validation-tuned adaptive candidate-budget beam search that preserves the original folded-subgraph and relaxation mechanisms. - We provide a Java implementation with deterministic preprocessing, restartable output, cached local computations, per-seed metrics, and figures. - We evaluate every eligible seed in LFR6-LFR9, totaling 3,000 held-out seeds, and report both gains and regressions.

The exact graph realizations used by the original paper are not distributed with the paper supplied to this project. Accordingly, the experiments below use deterministic generated LFR graphs with the paper's published macro parameters. The results are implementation results on those realizations, not claims that the paper's original Table III-V numbers have been reproduced.

Background and Original FG Method

Local problem definition

Let G=(V,E) be an undirected graph and let s in V be a seed node. The goal is to estimate the hidden community C_s containing s using local graph access. The algorithm returns C_s. Evaluation compares the two sets using recall, precision, and F1:

$$R = |C_s \text{ intersection } C_s||C_s|, P = |C_s \text{ intersection } C_s||C_s|, F_1 = 2RPR+P.$$

Folded-subgraph construction

For a starting node v, the folding procedure initializes F=\v\ and R=N(v). At each step, it selects the candidate with the largest number of links into F. Ties are resolved by smaller degree and then by node ID. The candidate is tentatively added only when the

induced density remains at least :

$$(F)=2|E(F)||F|(|F|-1) >= .$$

The result is denoted by F(v).

Folding-community stage

Starting from a current community C, the original method constructs folded subgraphs for boundary candidates. It accepts a folded subgraph when at least half of its nodes overlap the current community:

$$|F(v) \text{ intersection } C| >= |F(v)|2.$$

The process repeats until a complete pass adds no node.

Modularity stage

For a community C, let e_in denote internal edges and e_out denote edges leaving the community. The local modularity ratio is:

$$M(C)=e_ine_out.$$

For a folded candidate F, the implementation follows the paper's local quantities. Let x count edges from new candidate nodes into C, let y count the contribution of alternative nodes that connect into C, and let q count internal edges among new nodes. If d is the sum of degrees of new candidate nodes, the boundary adjustment is:

$$z=d-x-q2.$$

The change used by the implementation is:

$$M = e_in+xe_out+z-y-M(C),$$

when the denominator is positive. The original method selects the candidate with maximum non-negative M and discards the remaining alternatives.

Relaxation stage

The relaxation score combines first- and second-order contact with the current community:

$$f_1(F,C)=|N(F) \text{ intersection } C||C|, f_2(F,C)=|NN(F) \text{ intersection } C||C|, f(F,C)=f_1(F,C)+ f_2(F,C).$$

The implementation uses =0.5 and accepts the best candidate when f(F,C)>= h. The full FG detector repeatedly applies folding-community expansion, modularity expansion, and relaxation until no progress is made.

Failure Analysis

Irreversible local choice

The modularity stage ranks candidates by immediate gain. Suppose candidate a has a slightly larger M than candidate b. The original method chooses a even if adding b would expose a sequence of dense candidates that eventually yields a better community. Because the current community changes after each addition, the discarded candidate cannot be reconsidered.

Capacity is not the primary issue

The repository includes pointwise logistic, MLP, Bernoulli-Bernoulli RBM, pairwise ranking, and shallow boosted-stump variants. The exploratory results did not show a consistent gain from simply increasing predictor complexity. This suggests that the main limitation is the irreversible search decision, rather than the absence of a more expressive candidate classifier.

Computational cost

The original and unrestricted beam implementations repeatedly build folded subgraphs and recompute local edge statistics. The final implementation uses incremental folding connection counts, cached folded subgraphs, cached community edge statistics, thread-local detector reuse, and resumable CSV output. These optimizations do not alter the mathematical acceptance rules.

Adaptive Beam-FG Method

Method overview

The proposed method is called Validation-Tuned Adaptive Candidate-Budget Beam-Search FG, abbreviated AdaptiveBeam. It retains a list of partial communities instead of a single modularity path. The state score is:

$$S(C)=_t=1\^d M_t+ p_t,$$

where p_t is a learned local compatibility probability and is a small secondary weight. The modularity gain remains the primary term.

Candidate budget

Before constructing an expensive folded subgraph, boundary candidates are ordered by: - decreasing number of links into the current community; - increasing graph degree; - increasing node identifier. Only the first B candidates are evaluated exactly. The budget is not chosen from held-out test data. It is selected on validation graphs from the set \16,32,64,128\.

Learned compatibility score

The scorer is logistic regression over nine local features: folded density, community overlap, first-order overlap, second-order overlap, modularity gain, external edge ratio, folded size, mean degree, and internal candidate ratio. Training uses LFR1-LFR5. Validation uses the first five eligible seeds in LFR3-LFR5. The selected scorer contributes only =0.10 to the path score; ground-truth labels are never used during held-out detection.

Algorithm 1. Adaptive candidate-budget beam expansion

```
[1]
current community C, graph G, density , width W,
depth D, budget B
Barrow\(C,0)\
C^*arrow C
d=1 to D
Narrowempty
each (C_i,S_i) in B
rank boundary candidates of C_i by local priority
each of the first B candidates v
construct F(v) using the original folding rule
F(v) adds a node and M(C_i,F(v))>=0
compute local features and p(v)
add (C_i union F(v),S_i+ M+0.10p(v)) to N
remove duplicate communities from N
retain the best W states by score
N=empty break
BarrowN
update C^* with the highest-scoring state
C^*
```

Experimental Design

Datasets

The implementation encodes the LFR parameter rows reported in the supplied paper. LFR1-LFR5 contain 200 nodes with average degree 7, maximum degree 15, and mixing coefficients from 0.1 to 0.5. LFR6 contains 1,000 nodes with average degree 15, maximum degree 50, and maximum community size 50. LFR7 uses the same graph scale with maximum community size 300. LFR8 and LFR9 contain 500 nodes with average degree 7 and maximum degree 15. All rows use the specified degree and community exponents and non-overlapping communities.

The supplied paper does not provide the complete original graph files and random seeds. The repository therefore uses deterministic generated realizations with the published macro parameters. This distinction is essential when comparing the present numbers with the paper's tables.

Protocol

The model is trained before held-out testing. Configuration search is limited to validation data. Once the candidate budget and path parameters are frozen, LFR6-LFR9 remain untouched until final evaluation. Every eligible node in the held-out graphs is used as a seed. The final evaluation contains 3,000 seeds per method.

Metrics

For each seed, recall, precision, F1, and runtime are recorded. The summary reports arithmetic means over seeds in each dataset. Paired comparisons use the same seed for the two methods. The report does not interpret a positive mean difference as proof of statistical significance without a separately specified confidence procedure.

Results

Validation

The validation sweep selected B=64:

Candidate-budget validation results.

Budget	Mean validation F1
16	0.339110
32	0.337700
64	0.369291
128	0.368839

All-seed comparison

Table (see tab:f1) reports the complete held-out comparison.

Mean F1 over all eligible held-out seeds.

Dataset	Seeds	FG	Full beam	AdaptiveBeam
LFR6	1000	0.754533	0.686188	0.686188
LFR7	1000	0.338073	0.586587	0.593505
LFR8	500	0.737770	0.851961	0.851974
LFR9	500	0.803206	0.872581	0.872581
All	3000	0.621031	0.711682	0.713990

Overall recall, precision, F1, and runtime.

Method	Recall	Precision	F1	ms/seed
FG	0.948242	0.538054	0.621031	1323.625
Full beam	0.889162	0.647175	0.711682	249.489
AdaptiveBeam	0.892300	0.648429	0.713990	180.392

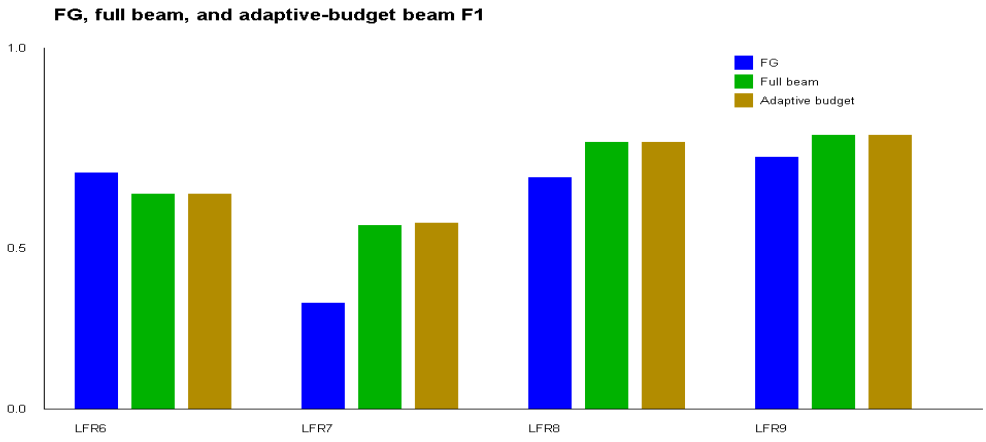

Figure 1. F1 comparison over LFR6-LFR9. The adaptive method preserves the beam improvement on LFR7-LFR9 while remaining close to unrestricted beam.

Discussion

The most important result is not only the aggregate mean. The dataset-wise pattern reveals the operating regime of the new method. LFR7 is the clearest case: the original greedy method obtains F1 0.338073, while unrestricted and adaptive beam obtain 0.586587 and 0.593505. This supports the hypothesis that greedy commitment is harmful when the local graph contains several plausible expansion routes. LFR8 and LFR9 show the same direction with smaller differences.

LFR6 is a counterexample. FG obtains F1 0.754533, while both beam variants obtain 0.686188. A beam search can preserve an attractive alternative that is not favorable for this graph regime, and the learned secondary score cannot guarantee correction. The method should therefore be presented as a robust search extension with regime-dependent behavior, not as a universally superior replacement.

The precision and recall averages also explain the F1 change. FG has higher recall overall but much lower precision. Beam methods reduce recall somewhat while improving precision substantially. The resulting balance produces the higher F1 values. This is consistent with the paper's motivation that local community methods must control boundary noise rather than simply absorb more neighbors.

Threats to Validity and Limitations

First, the original graph realizations and random seeds used by the paper are not available in this repository. The LFR results reproduce the published parameter regimes, not the authors' exact realizations. Second, the complete experiment is a benchmark study on four held-out LFR settings; it is not a substitute for evaluation on the paper's real-world networks. Third, the candidate budget was selected from a small validation sweep. A larger study could investigate budget stability across graph families. Fourth, runtime depends on the Java runtime, hardware, graph representation, and cache state. Finally, the reported paired counts are descriptive; a formal confidence interval or paired randomization test should be added before a publication claim about statistical significance.

Reproducibility

The source code, generated data, configuration, per-seed CSV, summary, and figure are stored in the public repository: center https://github.com/sankar1912/fg-based-local-community-detection-java center

The main commands are: javac -Xlint:all -d out *.java java -cp out foldedsubgraph.RigorousEvaluationTest java -cp out foldedsubgraph.AdaptiveBeamExperiment datasets/generated-lfr output/adaptive-beam-results-full 0

The final configuration is recorded in output/adaptive-beam-results-full/experiment-config.txt. The complete summary is in summary.csv, and the 9,000 per-seed rows are in per-seed.csv. The implementation also records validation results so that the selected budget can be audited.

Conclusion

This paper presented AdaptiveBeam, a validation-tuned adaptive candidate-budget beam-search extension of folded-subgraph local community detection. The method targets a specific weakness of the original FG algorithm: its irreversible selection of one immediate modularity winner. By retaining a small number of expansion paths and restricting expensive folding to a validated candidate budget, the method improves the overall F1 score and reduces runtime relative to unrestricted beam search in the complete evaluation.

Across 3,000 held-out seeds, AdaptiveBeam reaches mean F1 0.713990 compared with 0.621031 for FG. The result is encouraging but conditional: LFR6 remains a regression, and the graph realizations are generated from published LFR parameters rather than recovered from the authors' private experiment files. The appropriate conclusion is that adaptive multi-path expansion is a promising improvement and that irreversible greedy commitment is a measurable weakness, not that the new method is universally optimal.

References

M. Zhang and W. Bi, ``A local community detection method based on folded subgraph,'' IEEE Transactions on Knowledge and Data Engineering, vol. 37, no. 7, pp. 3869-3880, 2025.

A. Lancichinetti, S. Fortunato, and F. Radicchi, ``Benchmark graphs for testing community detection algorithms,'' Physical Review E, vol. 78, no. 4, Art. no. 046110, 2008.

M. E. J. Newman, ``Modularity and community structure in networks,'' Proceedings of the National Academy of Sciences, vol. 103, no. 23, pp. 8577-8582, 2006.

Y. Freund and R. E. Schapire, ``A decision-theoretic generalization of on-line learning and an application to boosting,'' Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119-139, 1997.